The brute force recursive solution has many redundant computations since we can end up with the same starting stone and jump size from many of our recursive paths. Hence, an obvious improvement is to memorise these computations. We can cache the result of the recursive function for the stone index and jump-size combination and if this same recursive call is made, we can return the cached result instead of the recursive function call.

As we compute the recursive solution, it becomes clear that at each stone, we are trying to find out what the possible jump sizes are from this stone. Once we have the set of jump sizes known for each stone, we can then check if there exists a stone at any of these jump sizes. If there exists a stone with that position, we can then update the new jump sizes for the newly reached stone. If we end up reaching the last stone during our traversal of all stones, then we have figured out that the frog can cross successfully. Else the frog cannot cross. We create a map which can remember the set of possible jump sizes associated from each stone's position. Hence, given a stone and its set of jump sizes, the DP solution computes for all reachable stones from the current stone, and their possible jump sizes. Given that this is a reachability problem, it is also possible to formulate this in terms of graph reachability. Given the starting stone and destination stone, the idea is to find out whether a possible path exists between them while obeying the constraints on the jump sizes. I will leave it to our readers to come up with the depth first search/breadth first search method of computing this reachability solution.

Another pattern that appears often in coding interviews is the sliding window pattern. Let us look at an example. Given a source string S and a target string t, write a function that can return the minimum length substring of S which contains all the characters in the target. The brute force approach is to enumerate all possible substrings of S, see which of them contain all characters in target t and choose the minimum length substring among these substrings. Instead, we can apply a sliding window approach to this problem.

Typically, a sliding window has a left pointer and a right pointer. We first expand the right pointer on the source string till the window includes all characters in target t. Now we have a valid substring containing all of the target characters, which is represented by the window. We can now move the left pointer forward to shrink the window so that we can reduce the length of the string. We keep repeating this operation over the complete source string and keep updating the minimum length substring among all the valid substrings we have encountered. I will leave it to our readers to write and test the actual code for this problem. The sliding window pattern is typically applicable to problems that are over the sequential single dimensional data structure such as an array or a string, though they can be applied with some modifications to matrix problems also. Here are a few sample problems for our readers to practice.

(1) Given an array of integers A of size N, find the contiguous sub-array with the minimum sum. A brute force approach would be to consider all sub-arrays (which is N^2), compute the sum for each of them and choose the minimum. Can you do better than that?
(2) Given a binary array containing 1s and 0s, find the length of the longest consecutive set of 1s in the array if you can replace k 0s with 1s. First assume that k is 1. Then solve this problem for any arbitrary 'k'.
(3) Given K sorted arrays, write a function to form a single sorted array from all these arrays.
(4) You are given an array of N integers. Write a function that computes the maximum absolute difference between the nearest smaller left element and the nearest smaller right element among all the array elements. If there is no left smaller element for a given array element, take the left smaller element as 0 (and do the same for the right smaller element).
(5) You are given an array of N integers, for which each array element represents the height of a histogram. Each of the histograms has unit widths. You are asked to find the area of the largest rectangle in the histogram. Remember that for any two histogram bars, the smallest bar lying between them decides the height of the rectangle.
(6) You are given an integer N. You need to find another integer M that has the same set of digits present in N but is greater in value than N. M should be the smallest such integer. If no such integer exists, return -1.

Feel free to reach out to me over LinkedIn/email if you need any help for your coding interview preparations. If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com.*

I request our readers to follow the public health guidelines and stay safe. Please don't worry if you are not able to focus fully on learning new technologies, programming, etc. These are difficult times. So first take care of your mental and physical health, help others wherever you can, and stay safe. Wishing all our readers happy coding until next month! Stay healthy and stay safe. END

By: Sandya Mannarswamy

The author is an expert in natural language processing and is currently working as an independent researcher. Her interests include natural language processing, machine learning and AI.